

Grundlagen der Informatik (GDI)

Vorlesung – Hochschule Mannheim

Aktivitätsdiagramme



Prof. Thomas Smits

Diese Materialien sind ausschließlich für den persönlichen Gebrauch durch Besucher der Vorlesung Grundlagen der Informatik (GDI) an der Technischen Hochschule Mannheim gedacht. Jede andere Nutzung ist ohne vorherige Zustimmung des Autors nicht zulässig.

Stand 2025-11-19

Technische Hochschule
mannheim

Inhaltsverzeichnis

1	Einführung	1
1.1	Motivation [5]	1
1.2	Symbole [6]	2
1.3	Regeln [7]	2
1.4	Aktionen [8]	3
1.5	Sequenz [9]	3
2	Entscheidungen	4
2.1	Entscheidungsknoten [11]	4
2.2	Else-Zweig [12]	4
2.3	Zusammenführungsknoten [13]	5
2.4	Beispiel zu Entscheidungsknoten [14]	6
3	Schleifen	7
3.1	Schleifen [16]	7
3.2	Beispiel: Schleife [18]	8
3.3	Zählschleifen [19]	8
3.4	Beispiel: Zählschleife [21]	9
3.5	Beispiel: Euklidischer Algorithmus [22]	9
4	Unterprogramme	11
4.1	Motivation [25]	11
4.2	Darstellung eines Unterprogramms [27]	12
4.3	Aufruf eines Unterprogramms [28]	12
	Index	i

Kapitel 1

Einführung

1.1 Motivation [5]

- **Aktivitätsdiagramm** (activity diagram) ist grafische Notation für
 - ▶ Algorithmen
 - ▶ Geschäftsprozesse
- standardisierte Symbole in **UML** (unified modeling language)
 - ▶ müssen eingehalten werden
 - ▶ gilt auch für Darstellung der Ecken und Verwendung von Klammern
- beschreibt Kontrollfluss
 - ▶ Aktivitäten: Was wird getan?
 - ▶ Kanten: Wie geht es weiter?

Aktivitätsdiagramm

UML

Bevor wir mit der Programmierung unter Zuhilfenahme einer Programmiersprache beginnen, sollen die Grundlagen algorithmischen Denkens anhand von Aktivitätsdiagrammen eingeführt werden. Sie stellen eine standardisierte, graphische Notation zur Verfügung. Aktivitätsdiagramme sind eine von vielen Möglichkeiten, die Abläufe in einem Programm darzustellen. Andere Notationen sind z. B.

- **Struktogramm** (auch **Nassi-Shneiderman-Diagramm** genannt) nach DIN 66261
- **Programmablaufplan** nach DIN 66001
- **Jackson-Diagramm**

Struktogramm

Nassi-Shneiderman-Diagramm

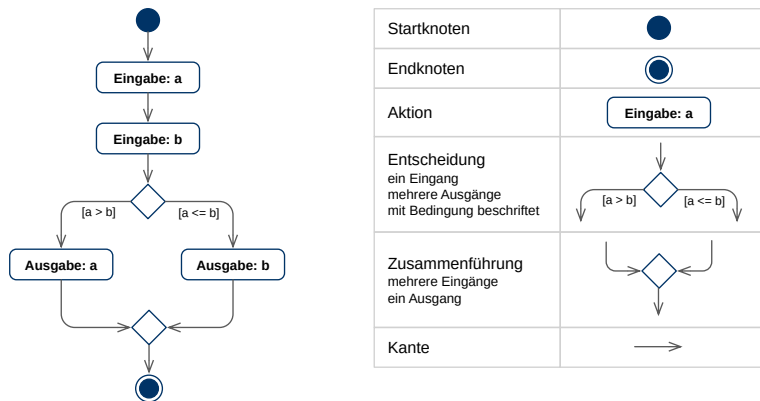
Programmablaufplan

Jackson-Diagramm

In welcher Notation man ein Programm darstellt ist weitgehend egal, da sich die einzelnen Diagramme ineinander überführen lassen. Die Wahl ist hier auf die Aktivitätsdiagramme gefallen, da sie zum UML-Standard gehören, der am weitesten verbreitet ist. Zwar brauchen Aktivitätsdiagramme mehr Platz als Struktogramme, sind aber die modernere Notation.

Die Unified Modeling Language ist eine grafische Modellierungssprache für Softwaresysteme. Sie wird von der Object Management Group (OMG) entwickelt und ist als ISO/IEC 19505 standardisiert.

1.2 Symbole [6]



Symbole des Aktivitätsdiagramms

Ein Aktivitätsdiagramm kommt mit einigen wenigen Elementen aus. Wie bei den meisten graphischen Darstellungen unterscheidet man zwischen:

- **Knoten (nodes)** stellen bestimmte Aktionen dar (aktiv)
- **Kanten (edges)** verbinden Knoten untereinander (passiv) und repräsentieren Beziehungen zwischen Knoten

Knoten

Kanten

In jedem Diagramm gibt es *genau einen* Startknoten, der den Anfang des Ablaufes darstellt. Es gibt *mindestens einen* Endknoten, an dem der Ablauf endet.

Innerhalb des Diagramms werden Aktionen und Entscheidungen dargestellt. Zu einem Zeitpunkt wird bei einer Entscheidung immer nur einer der Zweige benutzt. Wenn sich das Diagramm an einem Entscheidungsknoten aufgeteilt hat, werden diese an einem Zusammenführungsknoten wieder vereinigt.

1.3 Regeln [7]

- genau ein *Startknoten*
 - ▶ soll immer oben stehen (Ablauf von oben nach unten)
 - ▶ hier beginnt der Algorithmus
- mindestens ein *Endknoten*
 - ▶ soll unten stehen
 - ▶ jeder Pfad durch das Aktivitätsdiagramm (Ablauf) muss mit Endknoten enden
 - ▶ der Algorithmus *terminiert* am Endknoten
- nicht erlaubt
 - ▶ im Nirwana endende Kante
 - ▶ Aktion ohne ausgehende Kante, d. h. ohne Folgeaktion

Die Regeln für Aktivitätsdiagramme sind weitgehend selbsterklärend. Wichtig ist, dass die Diagramme von oben nach unten geschrieben werden, sodass man den Ablauf leicht erkennen kann. Deswegen steht der *einzigste* Startknoten immer oben.

Die Endknoten (häufig hat man nur einen Endknoten) stehen unten. Da der Ablauf am Endknoten endet, und jede Aktivität enden muss, endet jeder mögliche Weg durch das Diagramm irgendwann an einem Endknoten.

1.4 Aktionen [8]

Aktionen repräsentieren einen *einzigsten* Verarbeitungsschritt

Aktionen

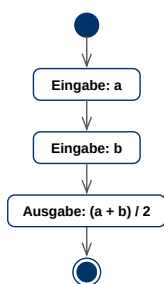
- haben mindestens einen Eingang
- haben mindestens einen Ausgang
- genau eine Aktion pro Rechteck
- müssen mit einer Tätigkeit beschriftet sein, z. B.
 - ▶ *Eingabe* einer Zahl oder einer Zeichenfolge (String) in eine Variable
z. B. *Eingabe*: `a`
 - ▶ *Ausgabe* einer Variablen oder eines berechneten Werts
z. B. *Ausgabe*: `a + b`
 - ▶ *Zuweisung* eines berechneten Werts an eine Variable
z. B. `mittelwert = (a + b) / 2`

1.5 Sequenz [9]

Sequenz (sequence)

Sequenz

- Abfolge von mehreren Aktionen ohne Verzweigungen
- Wird exakt von oben nach unten abgearbeitet



Sequenz von Aktionen

Fast alle Abläufe benötigen mehr als eine Aktion. Da pro Aktionsknoten nur ein Verarbeitungsschritt angegeben werden kann, benötigt man häufig eine Abfolge von Aktionen, die *Sequenz*.

Die Aktionen werden hier einfach untereinander geschrieben und mit Kanten verbunden. Da keine Verzweigung innerhalb der Sequenz erfolgt, werden immer alle Aktionen der Sequenz ausgeführt oder eben keine.

Kapitel 2

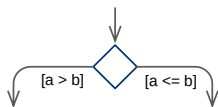
Entscheidungen

2.1 Entscheidungsknoten [11]

Entscheidungsknoten stellen eine Verzweigung dar

Entscheidungsknoten

- genau eine Eingangskante
- mindestens zwei Ausgangskanten
 - ▶ jede Kante mit einer Bedingung beschriftet
 - ▶ nur eine Bedingung darf zutreffen



Verzweigungsknoten

Die allermeisten Probleme lassen sich nur dann lösen, wenn man Entscheidungen bei der Lösung treffen kann. In Aktivitätsdiagrammen werden Fallunterscheidungen mit Hilfe von **Entscheidungsknoten** realisiert. An einem Entscheidungsknoten teilt sich das Diagramm in zwei oder mehr Äste auf. An den einzelnen Ästen steht, unter welcher **Bedingung** (condition) sie benutzt werden.

Bedingung

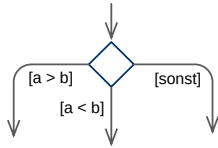
In dem Beispiel wird der linke Ast dann benutzt, wenn a größer als b ist. Der rechte Ast wird gewählt, wenn a kleiner oder gleich b ist.

2.2 Else-Zweig [12]

Wenn bei einem **Entscheidungsknoten** keine Bedingung zutrifft, kann man einen **Else-Zweig** verwenden

Else-Zweig

- Wird genommen, wenn keine Bedingung zutrifft
- Beschriftung mit **[else]** oder **[sonst]**



Verzweigungsknoten mit else-Zweig

Häufig gibt es bei Verzweigungen den Fall, dass verschiedene Fälle unterschieden werden sollen und wenn *keine* der Bedingungen zutrifft, soll auch eine bestimmte Aktion erfolgen. Diesen Fall nennt man auch den Else-Zweig.

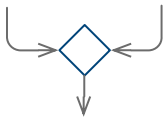
2.3 Zusammenführungsknoten [13]

Nach einer Verzweigung muss Ablauf oft wieder zusammengeführt werden

Zusammenführungsknoten

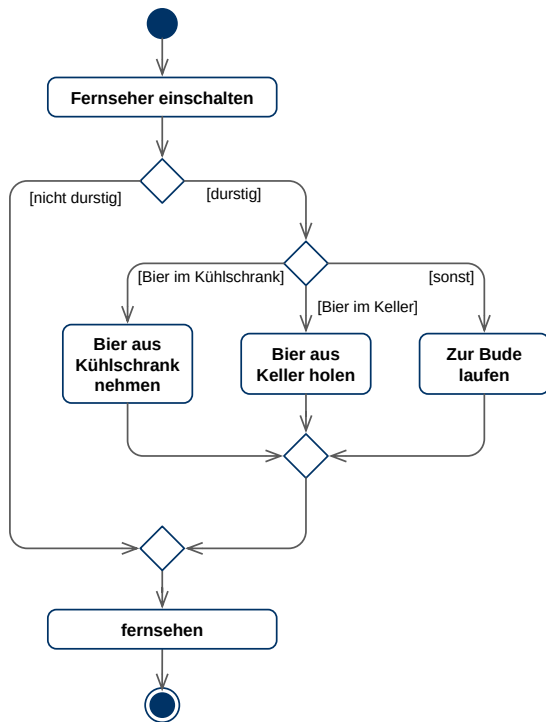
Zusammenführungsknoten

- mindestens zwei Eingangskanten
- genau eine Ausgangskante



Zusammenführungsknoten

2.4 Beispiel zu Entscheidungsknoten [14]



Beispiel für Entscheidungsknoten

Das Beispiel zeigt ein Aktivitätsdiagramm mit zwei Entscheidungsknoten für einen gemütlichen Fernsehabend. Der erste Knoten testet, ob der Zuschauer durstig ist. Ist dem nicht der Fall, kommt der zweite Entscheidungsknoten gar nicht mehr zum Zuge. Der zweite Knoten enthält einen Else-Zweig für den Fall, dass weder im Keller noch im Kühlschrank ein Bier vorhanden ist.

Kapitel 3

Schleifen

3.1 Schleifen [16]

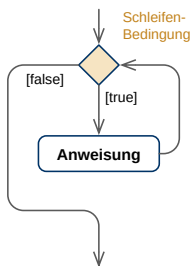
Manche Operationen muss man wiederholen, bis eine bestimmte Bedingung eingetreten ist bzw. solange eine Bedingung noch nicht eingetreten ist.

Löse solange Übungsaufgaben, bis der Stoff verstanden wurde

Schleifen (loops) bestehen aus

- Bedingung, die Wiederholung kontrolliert (**Schleifenbedingung**)
- Anweisungen, die wiederholt werden sollen

Im Aktivitätsdiagramm erkennt man Schleifen an einem Pfeil, der von unten nach oben verläuft (**Rückwärtssprung**)



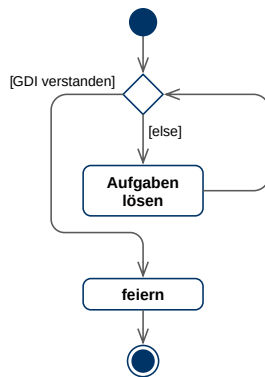
Schleife

Schleifen

Schleifenbedingung

Rückwärtssprung

3.2 Beispiel: Schleife [18]



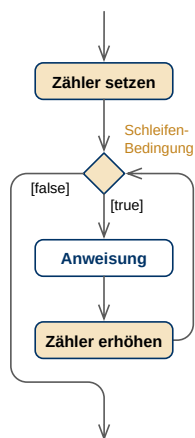
Beispiel zur Schleife

3.3 Zählschleifen [19]

Soll die Schleife eine bestimmte Anzahl von Durchläufen machen, verwendet man eine **Zählschleife**

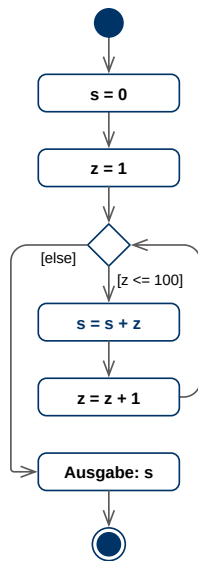
Zählschleife

- Variable speichert Anzahl der Durchläufe
- Variable wird bei jedem Durchlauf um einen bestimmten Wert erhöht (meist um eins)
- Schleife bricht ab, wenn gewünschte Anzahl der Durchläufe erreicht ist



Zählschleife im Aktivitätsdiagramm

3.4 Beispiel: Zählschleife [21]

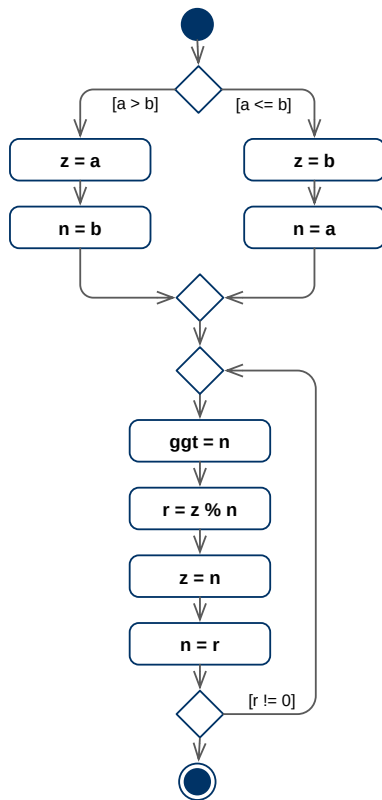


Beispiel zur Zählschleife

3.5 Beispiel: Euklidischer Algorithmus [22]

Gesucht ist der größte gemeinsame Teiler zweier Zahlen a, b $\text{ggT}(a, b)$

1. Man dividiert die größere durch die kleiner Zahl mit Rest und erhält den Quotienten q_0 und den Rest r_0
2. Ist der $r_0 = 0$, dann ist man fertig. b ist der GGT ($\text{ggT}(a, b) = b$)
3. Wenn $r_0 \neq 0$ ist, dann dividiert man b durch r_0 und erhält Quotienten q_1 und Rest r_1
4. Ist $r_1 = 0$, dann ist man fertig. r_0 ist der GGT ($\text{ggT}(a, b) = r_0$)
5. Wenn $r_1 \neq 0$ ist, dann wiederholt man die Schritte, bis man einen Rest von 0 erhält. Der Rest des vorhergehenden Schritts (r_{i+1}) ist dann der GGT



Euklidischer Algorithmus

Kapitel 4

Unterprogramme

4.1 Motivation [25]

Aktivitätsdiagramme werden schnell sehr umfangreich

- passen nicht mehr auf eine Seite
- werden unübersichtlich

Gewisse Aktivitäten kommen immer wieder vor, z. B. Suche nach dem größten Element in einer Reihe von Zahlen

Lösung

- man erstellt eigene Diagramme für Funktionen **Unterprogramme** (subprogram)
- man „ruft“ die Funktionen aus den anderen Diagrammen auf
- über **Parameter** können Daten übergeben werden
- über **Rückgabewerte** (return values) können Daten zurückfließen

Unterprogramme

Parameter

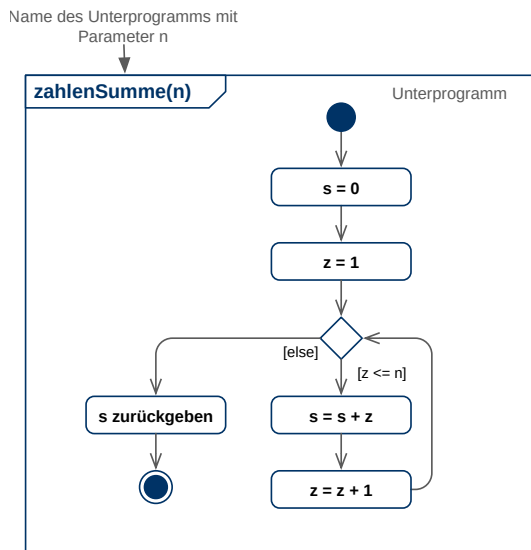
Rückgabewerte

Das Aufteilen größerer Probleme in kleinere ist eine der wichtigsten Methoden der Informatik („teile und herrsche“). Nur so besteht die Möglichkeit, komplexe Aufgaben noch mit Hilfe von Programmen zu lösen. Außerdem erlaubt diese Aufteilung, bereits erstellte Lösungen wiederzuverwenden. Die Aufteilung erfolgt, indem man Diagramme für **Unterprogramme** erstellt.

Damit das Unterprogramm flexibel bleibt, d. h. auf identische Probleme mit unterschiedlichen Daten angewendet werden kann, kann man dem Unterprogramm mit Hilfe von **Parametern** Daten mitgeben. Dieses Vorgehen ist üblich und auch aus der Mathematik bekannt. Auch hier kann man z. B. die Sinus-Funktion mit unterschiedlichen Winkeln aufrufen. Das Ergebnis des Unterprogramms wird dann als **Rückgabewert** an das **Hauptprogramm** (main program) zurückgegeben.

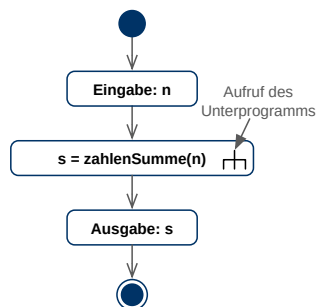
Hauptprogramm

4.2 Darstellung eines Unterprogramms [27]



Unterprogramm

4.3 Aufruf eines Unterprogramms [28]



Aufruf des Unterprogramms

Beim Aufruf des Unterprogramms kann der Rückgabewert einer Variable des Hauptprogramms zugewiesen werden, damit dieses den Wert weiterverwenden kann. Im Beispiel wird eine Zahl n vom Benutzer eingelesen. Mit dieser Zahl wird das Unterprogramm `zahlenSumme` aufgerufen. Das Ergebnis speichert das Hauptprogramm in der Variable s , die es dann im nächsten Schritt ausgibt.

Index

Aktionen, 3
Aktivitätsdiagramm, 1

Bedingung, 4

Else-Zweig, 4
Entscheidungsknoten, 4

Hauptprogramm, 11

Jackson-Diagramm, 1

Kanten, 2
Knoten, 2

Nassi-Shneiderman-Diagramm, 1

Parameter, 11
Programmablaufplan, 1

Rückgabewerte, 11
Rückwärtssprung, 7

Schleifen, 7
Schleifenbedingung, 7
Sequenz, 3
Struktogramm, 1

UML, 1
Unterprogramme, 11

Zusammenführungsknoten, 5
Zählschleife, 8